# LAB 7 | FUZZ TESTING

## Input Validation Bugs in the Income Tax Application

Student: SARTHKI VACHHANI
SRN: PES1UG24CS422

### Application under test

The Lab 6 income-tax program, with its three functions:
taxable_income, tax_due, and refund_status.

Hypothesis fuzz tests exercised valid rules and malformed inputs.
The initial run exposed eight missing validation behaviors.
The corrected program validates whole-number and boolean inputs,
rejects invalid negative amounts and ages, and passes all 11 tests.

This consolidated report includes the four requested run evidence
panels and the complete final Python source files.

# 1. Original application execution

Rendered evidence panel from the saved command output transcript.

```
ORIGINAL LAB 6 APPLICATION RUN

$ python processor_buggy.py
Taxable income: 925000
Tax due: 32500
Refund status: refund
```

## 2. Failing cases found by fuzz testing

Rendered evidence panel from the saved command output transcript.

HYPOTHESIS FOUND INPUT VALIDATION FAILURES

```
$ python -m pytest test_processor.py -q
FAILED test_processor.py::test_taxable_income_rejects_negative_investments - ...
FAILED test_processor.py::test_taxable_income_rejects_non_integer_gross - Fai...
FAILED test_processor.py::test_tax_due_rejects_negative_age - Failed: DID NOT...
FAILED test_processor.py::test_tax_due_rejects_non_integer_taxable_income - F...
FAILED test_processor.py::test_refund_status_rejects_negative_tax_paid - Fail...
FAILED test_processor.py::test_refund_status_rejects_negative_tax_due - Faile...
FAILED test_processor.py::test_refund_status_requires_boolean_filing_flag - F...
FAILED test_processor.py::test_refund_status_rejects_non_integer_tax_paid - F...
8 failed, 3 passed, 1 warning in 4.71s
```

# 3. Corrected application execution

Rendered evidence panel from the saved command output transcript.

```
CORRECTED LAB 6 APPLICATION RUN

$ python processor.py
Taxable income: 925000
Tax due: 32500
Refund status: refund
```

# 4. Final successful test execution

Rendered evidence panel from the saved command output transcript.

```
FINAL HYPOTHESIS FUZZ TESTS

$ python -m pytest test_processor.py -q
...........                                                              [100%]
11 passed, 1 warning in 3.51s
```

# Appendix A: Corrected application source

processor.py | lines 1-48

```
"""Hardened Lab 6 income tax application used as the Lab 7 fuzzing target."""
def _require_integer(value, name):
    """Reject floats and booleans where the tax rules require whole numbers."""
    if not isinstance(value, int) or isinstance(value, bool):
        raise TypeError(f"{name} must be an integer")
def taxable_income(gross, regime, investments):
    _require_integer(gross, "gross")
    _require_integer(investments, "investments")
    if not isinstance(regime, str):
        raise TypeError("regime must be a string")
    if gross < 0:
        raise ValueError("gross must be nonnegative")
    if investments < 0:
        raise ValueError("investments must be nonnegative")
    if regime == "new":
        deduction = 75000
    elif regime == "old":
        if investments > 150000:
            investments = 150000
        deduction = 50000 + investments
    else:
        raise ValueError("unknown regime")
    taxable = gross - deduction
    if taxable < 0:
        return 0
    return taxable
def tax_due(taxable, age):
    _require_integer(taxable, "taxable")
    _require_integer(age, "age")
    if taxable < 0:
        raise ValueError("invalid amount")
    if age < 0:
        raise ValueError("age must be nonnegative")
    if age >= 80:
        exempt = 500000
    elif age >= 60:
        exempt = 300000
    else:
        exempt = 250000
    if taxable <= exempt:
        return 0
    slab = taxable - exempt
```

# Appendix A: Corrected application source (continued)

processor.py | lines 49-81

```
    if slab <= 250000:
        return slab * 5 // 100
    return 12500 + (slab - 250000) * 20 // 100
def refund_status(tax_paid, tax_due_amount, filed_on_time):
    _require_integer(tax_paid, "tax_paid")
    _require_integer(tax_due_amount, "tax_due_amount")
    if not isinstance(filed_on_time, bool):
        raise TypeError("filed_on_time must be a boolean")
    if tax_paid < 0 or tax_due_amount < 0:
        raise ValueError("tax amounts must be nonnegative")
    difference = tax_paid - tax_due_amount
    if difference > 0:
        if filed_on_time:
            return "refund"
        return "refund delayed"
    elif difference < 0:
        if filed_on_time:
            return "pay balance"
        return "pay balance with penalty"
    return "settled"
def main():
    """Run a short demonstration with ordinary Lab 6 inputs."""
    print("Taxable income:", taxable_income(1000000, "new", 0))
    print("Tax due:", tax_due(600000, 30))
    print("Refund status:", refund_status(15000, 12500, True))
if __name__ == "__main__":
    main()
```

# Appendix B: Hypothesis fuzz tests

test_processor.py | lines 1-48

```
"""Hypothesis fuzz tests for the three Lab 6 income tax functions."""
import pytest
from hypothesis import example, given, strategies as st
from processor import refund_status, tax_due, taxable_income
@given(
    gross=st.integers(min_value=0, max_value=5_000_000),
    regime=st.sampled_from(["new", "old"]),
    investments=st.integers(min_value=0, max_value=500_000),
)
def test_taxable_income_matches_regime_rule(gross, regime, investments):
    if regime == "new":
        deduction = 75_000
    else:
        deduction = 50_000 + min(investments, 150_000)
    assert taxable_income(gross, regime, investments) == max(0, gross - deduction)
@given(investments=st.integers(max_value=-1))
@example(investments=-1)
def test_taxable_income_rejects_negative_investments(investments):
    with pytest.raises(ValueError, match="investments must be nonnegative"):
        taxable_income(1_000_000, "old", investments)
@given(gross=st.floats(allow_nan=False, allow_infinity=False))
@example(gross=1.5)
def test_taxable_income_rejects_non_integer_gross(gross):
    with pytest.raises(TypeError, match="gross must be an integer"):
        taxable_income(gross, "new", 0)
@given(
    taxable=st.integers(min_value=0, max_value=2_000_000),
    age=st.integers(min_value=0, max_value=120),
)
def test_tax_due_matches_exemption_and_slab_rules(taxable, age):
    exemption = 500_000 if age >= 80 else 300_000 if age >= 60 else 250_000
    slab = max(0, taxable - exemption)
    expected = slab * 5 // 100 if slab <= 250_000 else 12_500 + (slab - 250_000) * 20 //
 100
    assert tax_due(taxable, age) == expected
@given(age=st.integers(max_value=-1))
@example(age=-1)
```

# Appendix B: Hypothesis fuzz tests (continued)

test_processor.py | lines 49-96

```
49  def test_tax_due_rejects_negative_age(age):
50      with pytest.raises(ValueError, match="age must be nonnegative"):
51          tax_due(100_000, age)
54  @given(taxable=st.floats(allow_nan=False, allow_infinity=False))
55  @example(taxable=100_000.5)
56  def test_tax_due_rejects_non_integer_taxable_income(taxable):
57      with pytest.raises(TypeError, match="taxable must be an integer"):
58          tax_due(taxable, 30)
61  @given(
62      tax_paid=st.integers(min_value=0, max_value=2_000_000),
63      due=st.integers(min_value=0, max_value=2_000_000),
64      on_time=st.booleans(),
65  )
66  def test_refund_status_matches_payment_difference(tax_paid, due, on_time):
67      if tax_paid > due:
68          expected = "refund" if on_time else "refund delayed"
69      elif tax_paid < due:
70          expected = "pay balance" if on_time else "pay balance with penalty"
71      else:
72          expected = "settled"
73      assert refund_status(tax_paid, due, on_time) == expected
76  @given(tax_paid=st.integers(max_value=-1))
77  @example(tax_paid=-1)
78  def test_refund_status_rejects_negative_tax_paid(tax_paid):
79      with pytest.raises(ValueError, match="tax amounts must be nonnegative"):
80          refund_status(tax_paid, 0, True)
83  @given(due=st.integers(max_value=-1))
84  @example(due=-1)
85  def test_refund_status_rejects_negative_tax_due(due):
86      with pytest.raises(ValueError, match="tax amounts must be nonnegative"):
87          refund_status(0, due, True)
90  @given(on_time=st.integers())
91  @example(on_time=0)
92  def test_refund_status_requires_boolean_filing_flag(on_time):
93      with pytest.raises(TypeError, match="filed_on_time must be a boolean"):
94          refund_status(10_000, 9_000, on_time)
```

# Appendix B: Hypothesis fuzz tests (continued)

test_processor.py | lines 97-101

```
97  @given(tax_paid=st.floats(allow_nan=False, allow_infinity=False))
98  @example(tax_paid=1.5)
99  def test_refund_status_rejects_non_integer_tax_paid(tax_paid):
100     with pytest.raises(TypeError, match="tax_paid must be an integer"):
101         refund_status(tax_paid, 0, True)
```